Exercise 2 — A Complete Comparison of Multilevel Queue Scheduling with EDF and RM

General-purpose versus real-time scheduling: theory, analytic tests and measured results

Exercise 2 — MLQ / EDF / RM comparison
Course Advanced Operating Systems
Topic General-purpose versus real-time scheduling
Deliverables Bilingual report (DOCX + PDF), runnable Python code, diagrams
Repository github.com/kaspian021/Multilevel-Queue-Scheduling
Branch Exercise-2-MLQ-EDF-RM-Comparison
Evaluation discrete simulation plus exact analytic tests
Status complete — every number below comes from an actual program run

Table of contents

3 Abstract, keywords and reading map
4 Introduction and problem statement
5 Goals, research questions and method
6 Fundamentals (1): real-time versus general-purpose scheduling
7 Fundamentals (2): the task model and its assumptions
8 Earliest Deadline First (EDF)
9 Rate Monotonic (RM)
10 Deadline Monotonic and the role of MLQ in this comparison
11 Comparing fundamental properties (1): preemption, priority and deadlines
12 Comparing fundamental properties (2): metrics, predictability and fairness
13 Designing the fair comparison scenario
14 Scenario A: measured results
15 Scenario A: Gantt charts
16 Scenario A: the behaviour of MLQ
17 Scenario B: the high-pressure test
18 Scenario B: Gantt charts and decision walkthrough
19 The Liu and Layland bound versus the exact RM test
20 EDF schedulability and the processor-demand criterion
21 Randomised feasibility study
22 Scenario C: an aperiodic workload outside the real-time model
23 Scenario C: results and interpretation
24 Scenario D: behaviour under overload
25 Scenario D: Gantt charts and the design message
26 Aggregate comparison charts
27 Jitter and scheduling overhead
28 CPU utilisation across the four scenarios
29 Comprehensive comparison table
30 Analysis of the results
31 Advantages and disadvantages of each algorithm
32 Applications, limitations and assumptions
33 Conclusion
34 References
35 Appendix: outputs and requirement mapping

Abstract, keywords and reading map

Abstract

Comparing scheduling algorithms only makes sense when they are driven by the same workload, the same timing model and the same measurement code. This report implements Multilevel Queue (MLQ), Earliest Deadline First (EDF) and Rate Monotonic (RM) inside one shared discrete harness and evaluates them on four scenarios.

Beyond measurement, the exact analytic tests were implemented as well: response-time analysis for fixed priorities and the processor-demand criterion for EDF. In the high-pressure scenario with a utilisation of 0.9714, response-time analysis predicts that RM is infeasible and the simulator records exactly the three deadline misses that the analysis implies, while EDF runs without a single miss. A randomised study over 2329 task sets shows that in the utilisation band 0.90–1.00 only 17.8% of the sets are feasible under RM, against 100% under EDF.

Keywords

Real-time scheduling; EDF; RM; multilevel queue; deadline; period; Liu and Layland bound; response-time analysis; processor-demand criterion; predictability; fairness; starvation.

Reading map

- Sections 1–5: fundamentals, real-time versus general-purpose scheduling, task model.
- Sections 6–10: the three algorithms and their structural differences.
- Sections 11–16: the fair comparison scenario and the measured results.
- Sections 17–20: analytic tests and the randomised feasibility study.
- Sections 21–25: aperiodic and overload scenarios, the master table, conclusions.

Introduction and problem statement

Problem statement

The question of this exercise is: how does the multilevel queue algorithm introduced in exercise 1 compare with the two classical real-time algorithms EDF and RM? The short answer is that the three were designed for three different problems, so a direct comparison without a carefully defined workload is misleading.

Scope of the comparison

- General-purpose scheduling aims at average metrics, fairness and responsiveness.
- Real-time scheduling aims at guaranteeing every deadline, at every instant.
- EDF and RM belong to the second family; MLQ belongs to the first.
- A numerical comparison therefore requires a task set with shared parameters.

The usual trap

Methodological warning: Comparing these three algorithms while ignoring their different nature is like comparing a family car with a truck on the basis of top speed on one road. A number comes out, but the interpretation is not valid. In this report every claim is stated together with the scenario and the range over which it holds.

- One shared simulation harness for all three algorithms.
- Four scenarios: fair comparison, high pressure, aperiodic load and overload.
- Independent analytic tests to confirm the measured results.
- A comprehensive comparison table with the limitations stated explicitly.

Goals, research questions and method

Overall goal

To deliver a scientific, documented and reproducible comparison between MLQ and the two real-time algorithms EDF and RM, separating clearly what was measured from what was proved analytically.

Specific objectives

- Explain the difference between real-time and general-purpose scheduling.
- Present the formal periodic task model and its assumptions.
- State the Liu and Layland sufficient condition for RM and show that it is not necessary.
- Implement the exact RM test based on response-time analysis.
- Implement the exact EDF test based on the processor-demand criterion.
- Design a fair comparison scenario and justify why it is fair.
- Measure response, turnaround, waiting, utilisation, fairness and overhead.

Research questions

- Under what conditions does RM fail while EDF succeeds?
- Can MLQ replace a real-time algorithm on a real-time workload?
- How large is the price of predictability, measured in context switches?
- Which algorithm behaves best under overload?

Method

- Implement all three algorithms inside one discrete simulator.
- Define four scenarios with explicit parameters and a fixed horizon.
- Run everything automatically and store every output as text and JSON.
- Cross-check the measurements against independent analytic tests.
- Generate every figure from the real data and analyse it numerically.

Fundamentals (1): real-time versus general-purpose scheduling

General-purpose scheduling

- Goal: optimise average metrics and keep the processor busy.
- Input: a set of processes with arrival times and burst lengths.
- Key concept: average waiting time, turnaround time and response time.
- Success means: better averages, more fairness, adequate responsiveness.
- Failure means: slowness, not necessarily the violation of a specific timing commitment.

Real-time scheduling

- Goal: meet the deadline of every job, at every instant.
- Input: periodic tasks with a worst-case execution time, a period and a deadline.
- Key concept: schedulability, not average metrics.
- Success means: not a single deadline is missed.
- Failure means: a missed deadline, which in hard systems can be catastrophic.

Dimension	General-purpose	Real-time
Goal	better averages	guarantee the deadline
Success criterion	average metrics	schedulability
Evaluation method	measurement and simulation	exact analytic test
Role of averages	central	almost meaningless
Examples	FCFS, RR, MLQ	RM, EDF, DM

Table 1. The fundamental difference between the two families

Key point: A common mistake in student reports is to use average waiting time as evidence that one real-time algorithm beats another. In real-time scheduling a task set that misses one deadline has failed, however good its average is.

Fundamentals (2): the task model and its assumptions

Every task tau_i is described by the triple (C_i, T_i, D_i): C is the worst-case execution time, T the period and D the relative deadline. A job is released at every k*T and each job must complete by k*T + D.

```
tau_i = (C_i, T_i, D_i)
 U = sum_i (C_i / T_i)
H = lcm(T_1, T_2, ..., T_n)      # hyperperiod
```

Standard notation for periodic tasks

Analysis assumptions

- Single processor, preemptive scheduling.
- Independent tasks: no blocking, no resource sharing, no priority inversion.
- The worst-case execution time is known and constant.
- Deadlines are constrained: D is less than or equal to T.
- The context-switch cost is ignored by the analysis.
- There is no release jitter.

Why the assumptions matter

- As soon as one assumption is violated the analytic result is no longer guaranteed.
- With semaphore blocking, for example, a priority-inversion analysis is required.
- With release jitter, the jitter must be added to the deadline.
- In this project every assumption holds and each one is documented in the code.

Validity of simulation: Simulating one hyperperiod shows what happens on that horizon; it does not prove that the set is schedulable for all time. The proof comes only from the analytic tests.

Earliest Deadline First (EDF)

Definition

EDF is a dynamic-priority algorithm: at every instant the ready job with the earliest absolute deadline runs. The priority of a task is therefore not fixed; it rises as its deadline approaches.

```
priority_i(t) = d_i(t) = k * T_i + D_i
schedule:  choose the ready job with the smallest d_i(t)
```

The EDF decision rule

Properties

- Dynamic: the priority is recomputed from the absolute deadline at every instant.
- Optimal among dynamic-priority algorithms: if any algorithm can schedule a set, so can EDF.
- For implicit deadlines the schedulability condition U <= 1 is both necessary and sufficient.
- For constrained deadlines the exact test is the processor-demand criterion.
- Higher computational overhead, because priorities must be recomputed continuously.

The overload weakness

- Under overload EDF tends to spread the failure across all tasks.
- This behaviour, known as the domino effect, is considered a weakness by some designers.
- In this report that behaviour is measured in the overload scenario.
- The usual remedy is to combine EDF with a protection policy for critical tasks.

Rate Monotonic (RM)

Definition

RM is a fixed-priority algorithm: the priority follows the period, so the task with the shortest period gets the highest priority. The rationale is that a task which repeats more often is the more urgent one.

```
T_i < T_j   =>   priority_i > priority_j
U <= n * (2^(1/n) - 1)   =>   schedulable under RM  (sufficient only)
```

The RM priority rule and the Liu and Layland sufficient condition

Properties

- Static: the priority is fixed at design time and never changes.
- Simple and cheap to implement: no priority recomputation is needed.
- Optimal among fixed-priority algorithms: if any fixed-priority scheme works, so does RM.
- Predictable: high-priority behaviour is independent of the lower-priority tasks.
- Limitation: at high utilisation it can lose sets that EDF still schedules.

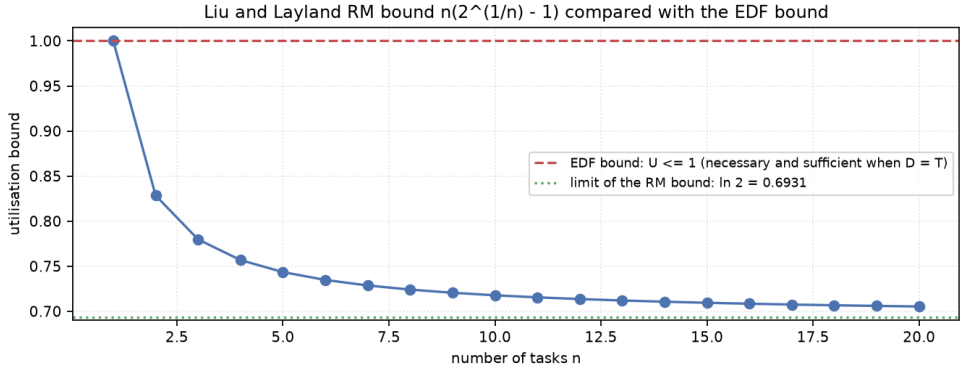


Figure 2. The Liu and Layland RM bound compared with the EDF bound and with ln 2

- For one task the bound is 1; it decreases towards ln 2 = 0.6931 as n grows.
- The EDF bound for implicit deadlines is always 1, independent of the number of tasks.
- The gap between the two curves is the price of the simplicity of fixed priorities.

Deadline Monotonic and the role of MLQ in this comparison

Deadline Monotonic (DM)

DM generalises RM to constrained deadlines: the priority follows the relative deadline. When D equals T, DM is identical to RM. DM is implemented in this project so that RM can be presented as a special case of a more general rule.

The position of MLQ

- MLQ was designed to classify processes, not to guarantee deadlines.
- To take part in the comparison, the queue level of every task must be derived from its parameters.
- In this project the queue level is stated explicitly in the scenario file.
- In the real-time scenarios the queues follow the period or the sensitivity of the task.
- Consequently MLQ behaves like RM in those scenarios; that is a finding, not a defect.

Property	EDF	RM / DM	MLQ
Priority type	dynamic	static	static
Priority basis	absolute deadline	period or relative deadline	queue level
Needs a deadline	yes	no (period only)	no
Main goal	meet deadlines	meet deadlines	separate workload classes
Analytic guarantee	yes	yes	no

Table 3. Structural differences between the three algorithms

Comparing fundamental properties (1): preemption, priority and deadlines

Preemptive versus non-preemptive

- All three algorithms are preemptive in this implementation: the running job may be stopped.
- In MLQ, preemption is always active between queues and depends on the policy inside a queue.
- In this comparison the MLQ intra-queue policy is FCFS so that MLQ stays comparable with RM.
- Choosing RR inside the queue increases overhead and improves response time.

Algorithm	When priority is set	Changes over time	Computational cost
EDF	every instant	yes	high
RM	design time	no	low
DM	design time	no	low
MLQ	queue assignment time	no (without aging)	low

Table 4. How each algorithm assigns priority

Deadline handling

- EDF uses the deadline directly: a nearer deadline means a higher priority.
- RM ignores the deadline and uses only the period.
- DM uses the relative deadline, but statically.
- MLQ has no notion of a deadline at all: a deadline only enters if the designer encodes it as a queue level.
- This difference is the main reason why no algorithm is universally better.

Comparing fundamental properties (2): metrics, predictability and fairness

Metric	Meaning in this comparison
Response time	completion minus the release instant of each job
Waiting time	not meaningful for periodic tasks, therefore not reported
Turnaround time	identical to response time for a periodic job
Utilisation	busy time divided by the whole simulation horizon
Jitter	standard deviation of the response time; a predictability measure
Context switches	a proxy for scheduling overhead

Table 5. Definitions of the measured metrics

Predictability

- Predictability means being able to compute the worst-case response time in advance.
- Under RM and DM, response-time analysis gives that number exactly.
- Under EDF, the processor-demand criterion decides schedulability but does not directly give response times.
- For MLQ there is no standard analytic treatment of deadlines.

Fairness and starvation

- Under MLQ and RM, low-priority tasks starve when the upper levels stay busy.
- Under EDF the priority is dynamic, so every task rises to the top as its deadline approaches.
- EDF is therefore theoretically fairer, but it spreads failure under overload.
- In MLQ, starvation can be contained with aging or time slicing, as shown in exercise 1.

Designing the fair comparison scenario

Why this scenario is fair

- All three algorithms run on exactly the same task set.
- The timing model is identical: discrete ticks, preemption allowed, free context switches.
- The horizon is a whole multiple of the hyperperiod, so no job is left half-finished.
- The MLQ queue level is derived from the task parameters, not chosen arbitrarily.
- No algorithm receives more information than the others.
- All metrics are computed by the same measurement code.

Task	C	T	D	Queue	Class
T1	1	5	5	1	sensitive
T2	2	8	8	1	sensitive
T3	2	10	10	2	medium
T4	3	20	20	3	background

Table 6. Scenario A: four periodic tasks with implicit deadlines

- Total utilisation is 0.800.
- The hyperperiod is 40 and the horizon is 120, that is, three complete hyperperiods.
- The total number of jobs over that horizon is 30.
- All deadlines are implicit: D equals T.

Scope of the result: This scenario only shows what happens on this particular workload. To generalise the result, more scenarios are needed or the analytic tests must be used.

Scenario A: measured results

Metric	EDF	RM	MLQ
Deadline misses	0	0	0
CPU utilisation	80.00%	80.00%	80.00%
Average response time	3.000	3.105	3.105
Jitter (standard deviation)	2.891	3.272	3.255
Context switches	54	57	54
Preemptions	9	12	9

Table 7. Scenario A results for the three algorithms (real program output)

- All three met every deadline, so on this workload they are equivalent in terms of correctness.
- EDF has the lower average response time: 3.000 against 3.105.
- EDF has the lower jitter: 2.891 against 3.272 for RM, that is, a steadier behaviour.
- MLQ behaved exactly like RM here, because the queue levels were aligned with the periods.
- Utilisation is 80.00% everywhere, which equals the offered load.

Test	Result	Interpretation
Liu and Layland bound	0.7568 < 0.800	inconclusive; the condition is sufficient only
RM response-time analysis	pass	RM is definitely schedulable
EDF processor-demand test	pass	EDF is definitely schedulable

Table 8. Matching the measurements against the analytic tests in scenario A

Teaching finding: This scenario is a good teaching example: the utilisation exceeds the Liu and Layland bound, yet the set is still schedulable under RM. The bound is not necessary.

Scenario A: Gantt charts

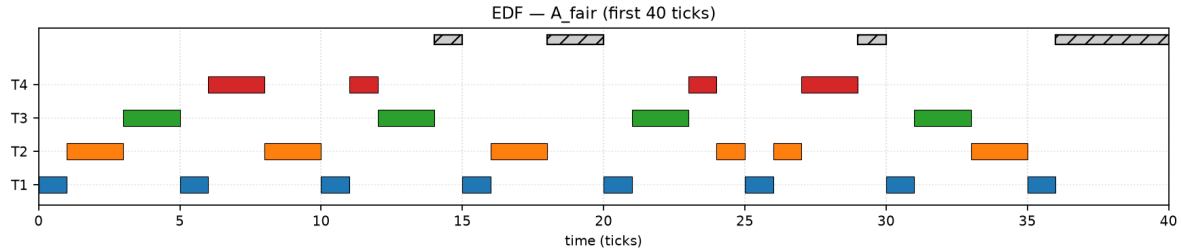


Figure 9. EDF Gantt chart in scenario A

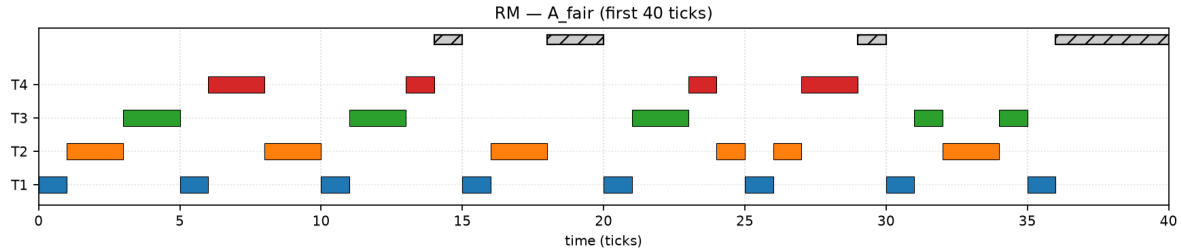


Figure 10. RM Gantt chart in scenario A; note the different placement of T3 and T4

Scenario A: the behaviour of MLQ

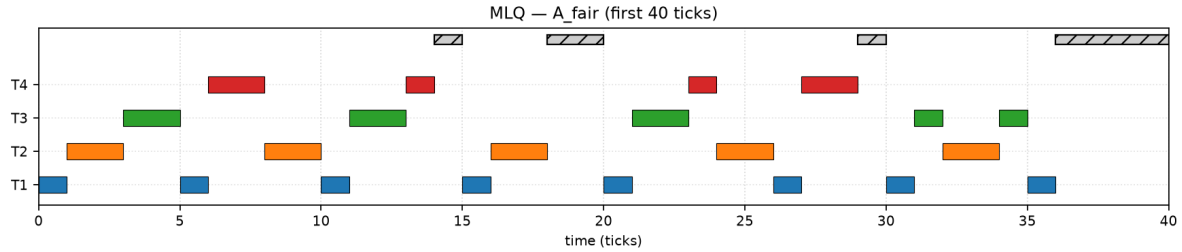


Figure 11. MLQ Gantt chart in scenario A; it coincides with RM here

- The queue levels were aligned with both the workload class and the period.
- When the queue order equals the period order, the fixed priority of MLQ reproduces RM.
- That alignment is not a coincidence: in practice designers usually treat faster tasks as more critical.
- Assigning the queue levels by a different criterion would give different results.
- The finding shows that the quality of MLQ depends on the assignment, not on the algorithm itself.

Scenario B: the high-pressure test

Task	C	T	D	Queue
A	2	5	5	1
B	4	7	7	2

Table 12. Scenario B: two tasks at a very high utilisation

- Total utilisation is 0.9714.
- The Liu and Layland bound for two tasks is 0.8284, so the utilisation exceeds it.
- The horizon is 105 ticks, that is, three complete hyperperiods.

Metric	EDF	RM	MLQ
Deadline misses	0	3	3
Average response time	3.833	4.000	4.000
Jitter	1.424	2.449	2.449
Context switches	36	48	48
Preemptions	3	15	15

Table 13. Scenario B results (real program output)

- Response-time analysis computes a worst-case response time of 8 for B, larger than its deadline of 7.
- The simulation shows exactly that: three jobs of B missed their deadline.
- EDF ran on the same workload without a single miss, because U is below 1.
- The difference in preemptions is striking: 3 against 15.

Scenario B: Gantt charts and decision walkthrough

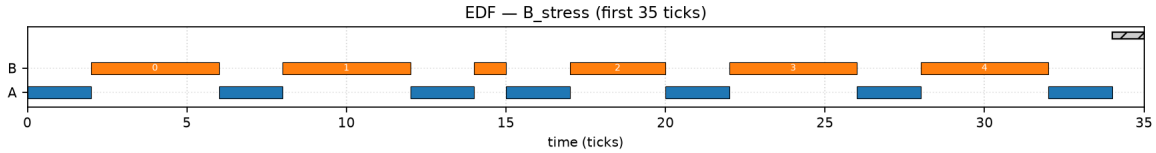


Figure 14. EDF Gantt chart in scenario B: no deadline is missed

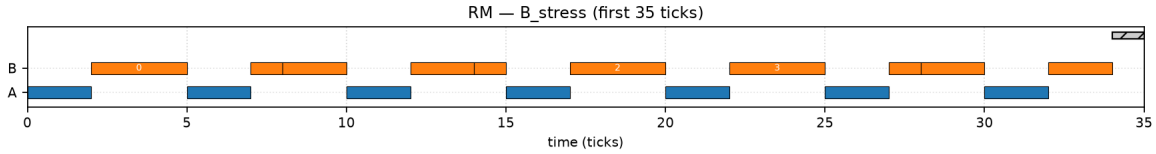


Figure 15. RM Gantt chart in scenario B: jobs of B miss their deadlines

- At t=0 both tasks are released; both algorithms run A first.
- At t=2 it is B's turn, but at t=5 the next job of A arrives.
- RM considers A more urgent and stops B with two ticks left.
- EDF sees the deadline of B#0 at t=7 and the deadline of A#1 at t=10, so it lets B#0 continue.
- Result: under RM, B#0 finishes at t=8 and misses its deadline of 7.

The Liu and Layland bound versus the exact RM test

```
                    U = sum (C_i / T_i)
U <= n * (2^(1/n) - 1)   =>   the task set is schedulable by RM
    n = 1 -> 1.0000     n = 2 -> 0.8284     n = 3 -> 0.7798
              n -> infinity :  U -> ln 2 = 0.6931
```

The Liu and Layland sufficient condition and its values

Why the condition is only sufficient

- The bound is derived from a worst-case arrangement, so many sets above it are still schedulable.
- Example from this project: scenario A has U = 0.800 against a bound of 0.7568 and is still RM-schedulable.
- Crossing the bound therefore means the exact test is needed, not that the set has failed.
- The exact test for fixed priorities is response-time analysis.

```
   R_i = C_i + sum over j in hp(i):  ceil(R_i / T_j) * C_j
          feasible  <=>  for every i:  R_i <= D_i
```

Response-time analysis for fixed priorities (Joseph and Pandya)

- Under the stated assumptions this test is exact for fixed-priority algorithms.
- The implementation iterates until convergence and stops as soon as R exceeds D.
- For scenario B it computes R_A = 2 and R_B = 8, which is greater than D_B = 7.

EDF schedulability and the processor-demand criterion

Implicit deadlines

If D equals T for every task then U <= 1 is both necessary and sufficient for EDF. This is a very powerful property: no more elaborate test is required.

Constrained deadlines

If some D is smaller than its T then U <= 1 is necessary but not sufficient. The exact test in that case is the processor-demand criterion.

```
dbf(t) = sum_i  max(0, floor((t - D_i) / T_i) + 1) * C_i
 schedulable by EDF  <=>  for every t > 0:  dbf(t) <= t
```

The demand bound function and the exact EDF criterion (Baruah, Howell and Rosier)

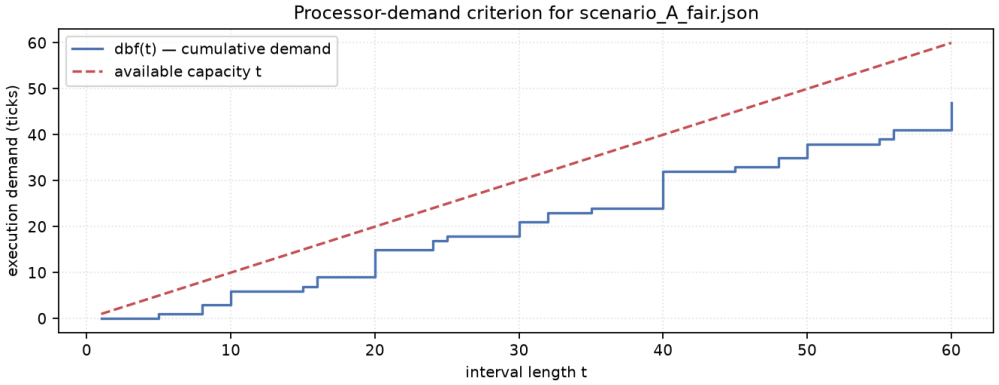


Figure 16. Demand bound function of scenario A against the available capacity

- Checking every value of t is impossible, so only points of the form k*T + D up to a valid bound are checked.
- This project uses the QPA bound and records the number of checked points in its output.
- In scenario C, which has constrained deadlines, the test checked 161 points.

Randomised feasibility study

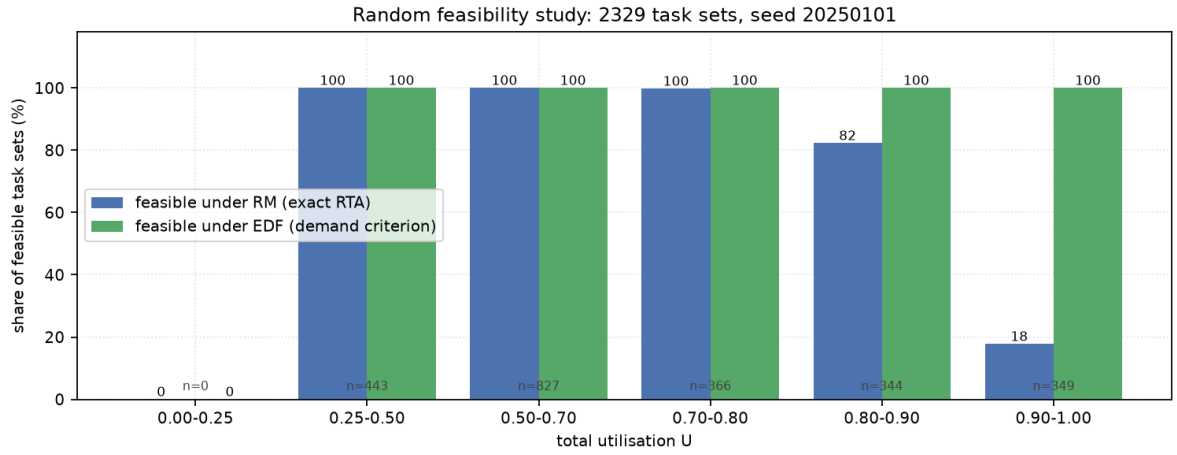


Figure 17. Share of feasible task sets under RM and under EDF, by utilisation band

Utilisation band	Task sets	Feasible under RM	Feasible under EDF
0.25 - 0.50	443	100.0%	100.0%
0.50 - 0.70	827	100.0%	100.0%
0.70 - 0.80	366	99.7%	100.0%
0.80 - 0.90	344	82.3%	100.0%
0.90 - 1.00	349	17.8%	100.0%

Table 18. Randomised study over 2329 task sets with seed 20250101

Method

- For every set the number of tasks is between 3 and 5 and the periods lie between 5 and 40.
- C is chosen so that the utilisation lands in the target band.
- RM is decided by response-time analysis, EDF by the utilisation condition (implicit deadlines).
- The random seed is fixed, so the whole study is reproducible.

Interpretation

- Up to a utilisation of 0.70 there is no visible difference between the two algorithms.
- From 0.80 upwards the gap opens quickly.
- In the top band only 17.8% of the sets are feasible under RM.
- The result confirms the theoretical optimality of EDF empirically.

Scenario C: an aperiodic workload outside the real-time model

The purpose of this scenario is to show that comparing real-time algorithms on a general-purpose workload is inherently unfair. It uses one-shot work items that have no period at all.

Task	C	Release	Deadline	Queue	Class
SYS1	3	0	6	1	system
INT1	4	1	12	1	interactive
BAT1	6	0	30	3	batch
INT2	3	2	13	2	interactive
BAT2	5	3	30	3	batch
INT3	2	8	14	1	interactive
BAT3	4	10	30	3	batch

Table 19. Scenario C: seven one-shot work items with synthetic deadlines

- Deadlines were invented artificially so that EDF can run at all.
- The period was set equal to the horizon so that RM can run at all.
- RM therefore has no rate information and degenerates into a first-come order.
- That is exactly why RM was not designed for general-purpose work.

Scenario C: results and interpretation

Metric	EDF	RM	MLQ
Deadline misses	0	3	0
Average response time	10.857	13.000	10.857
Jitter	7.221	4.320	7.493
Context switches	6	6	7
Preemptions	0	0	1

Table 20. Scenario C results (real program output)

- MLQ needed no deadline and no period at all: it decided by workload class and met the deadlines.
- EDF also succeeded, but its output depends on the quality of the invented deadlines.
- RM missed three deadlines because it had no rate information to work with.
- MLQ used one extra context switch because one inter-queue preemption occurred.
- The scenario shows that the algorithm must fit the nature of the workload.

Guiding principle: Whenever we have to invent parameters artificially in order to run a real-time algorithm, that is a sign the algorithm was not built for that problem. Numbers obtained under such conditions must not be used to rank the algorithms.

Scenario D: behaviour under overload

Metric	EDF	RM	MLQ
Deadline misses	31	10	10
CPU utilisation	100%	100%	100%
Average response time	18.200	11.400	11.400
Maximum response time	30	96	96
Jitter	6.975	20.612	20.612
Context switches	29	35	35

Table 21. Scenario D results at a utilisation of 1.233 (real program output)

Task	Priority	Misses under EDF	Misses under RM and MLQ
X	high	13 of 15	0 of 15
Y	medium	10 of 12	0 of 12
Z	low	8 of 10	10 of 10

Table 22. Distribution of the deadline misses per task in the overload scenario

- Under RM and MLQ the high-priority tasks are fully protected and the whole failure lands on Z.
- Under EDF the failure is spread over all tasks: that is the domino effect.
- The maximum response time under RM is 96 ticks: Z waited almost until the end of the horizon.
- The maximum response time under EDF is 30, so no task waited for an extreme period.

Scenario D: Gantt charts and the design message

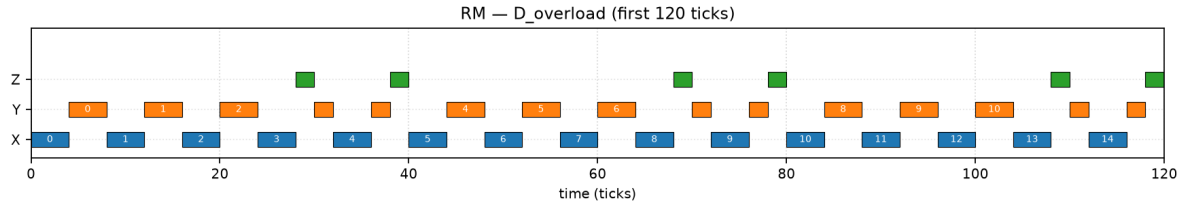


Figure 23. RM under overload: Z is squeezed between the higher-priority tasks

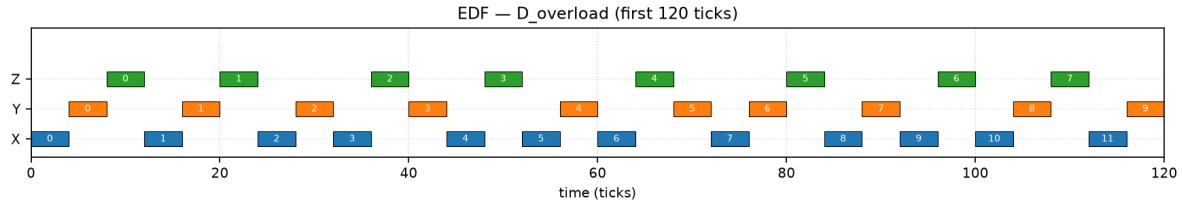


Figure 24. EDF under overload: execution is shared between all three tasks

- If the goal is to protect critical tasks, RM or MLQ are the better choice.
- If the goal is to avoid the catastrophic failure of one particular task, EDF behaves more evenly.
- In hard systems overload must be handled by admission control, not by the choice of algorithm.
- No algorithm can run a set with utilisation above 1 without missing deadlines.

Aggregate comparison charts

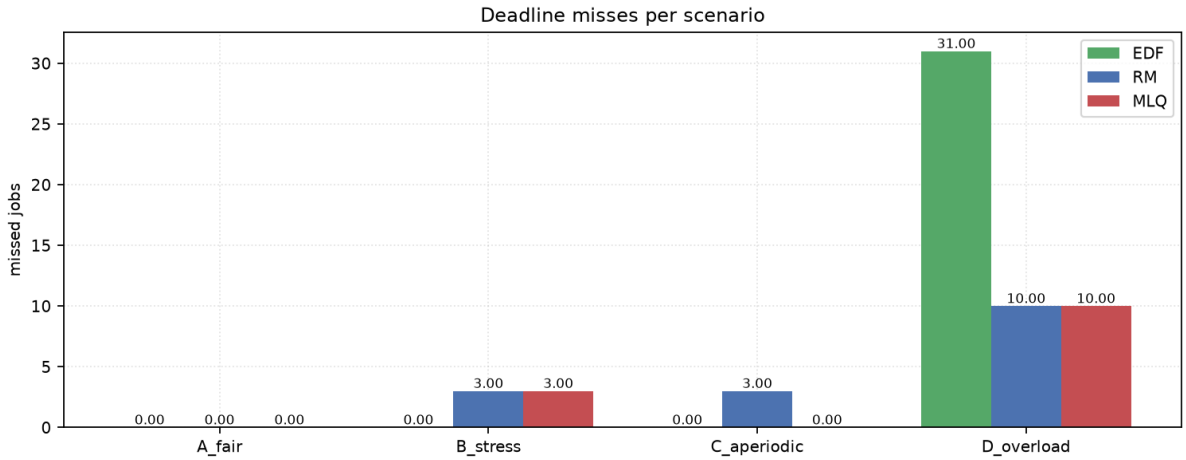


Figure 25. Deadline misses across the four scenarios

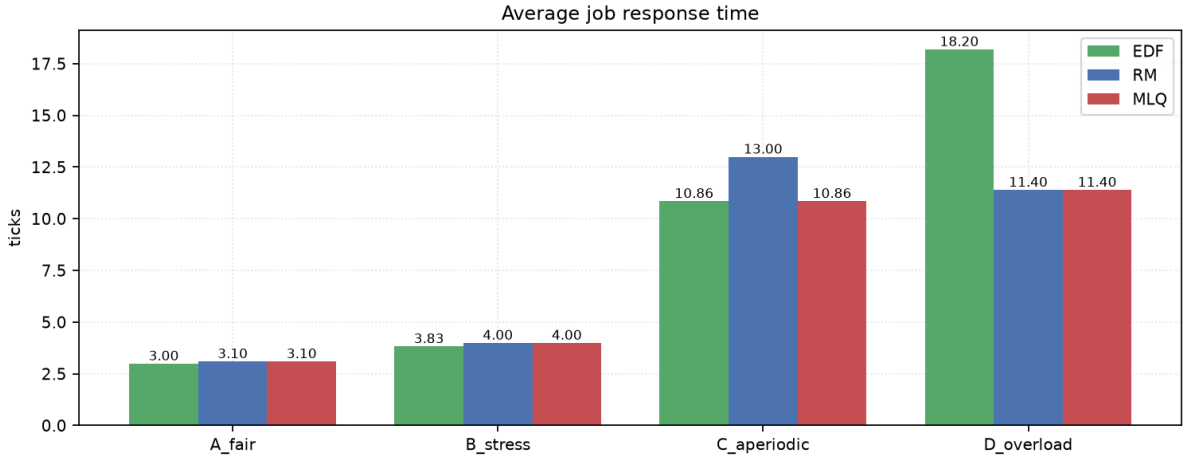


Figure 26. Average response time across the four scenarios

Jitter and scheduling overhead

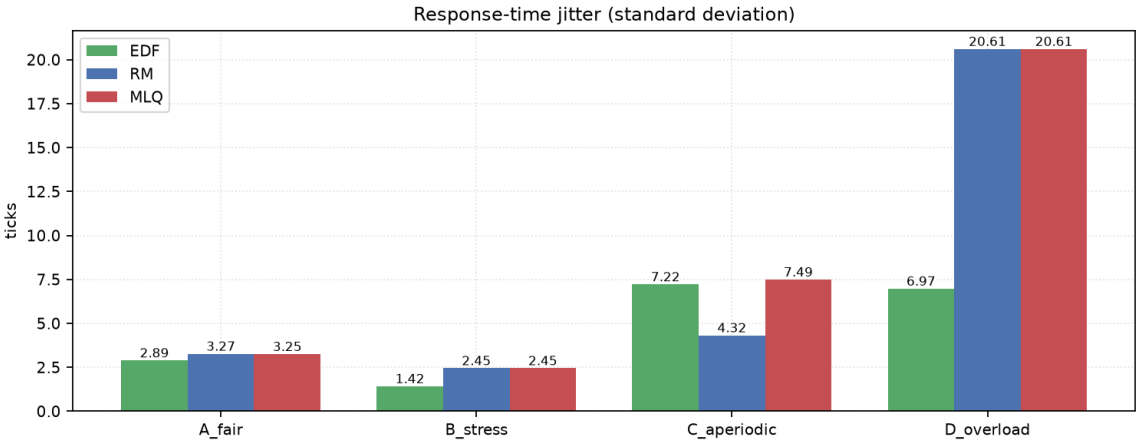


Figure 27. Response-time jitter (standard deviation) across the four scenarios

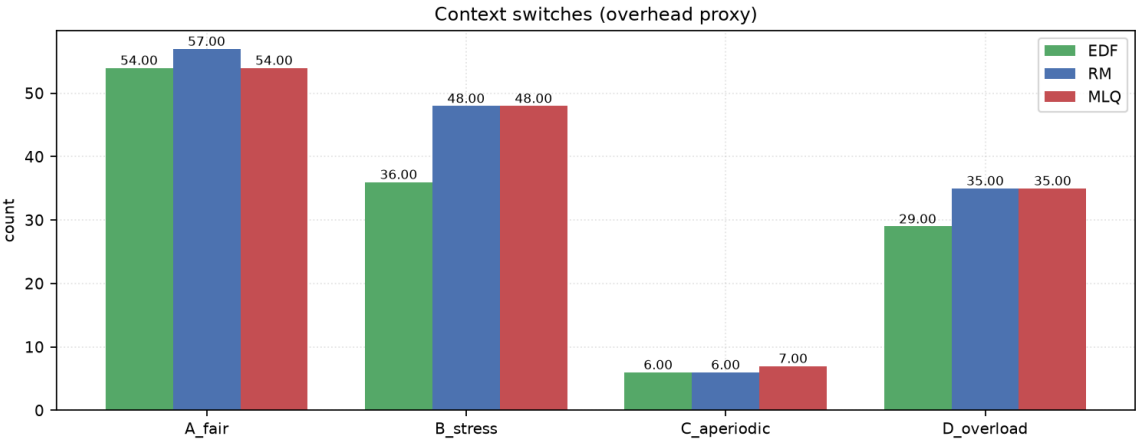


Figure 28. Context switches as a proxy for overhead

- In scenario A, EDF has the lowest jitter and RM the highest.
- In scenario B, EDF needed only 3 preemptions against 15 for RM: far less overhead.
- In scenario D the jitter of RM is 20.612, more than three times the 6.975 of EDF.
- Lower jitter means steadier and more predictable behaviour for sensitive tasks.

CPU utilisation across the four scenarios

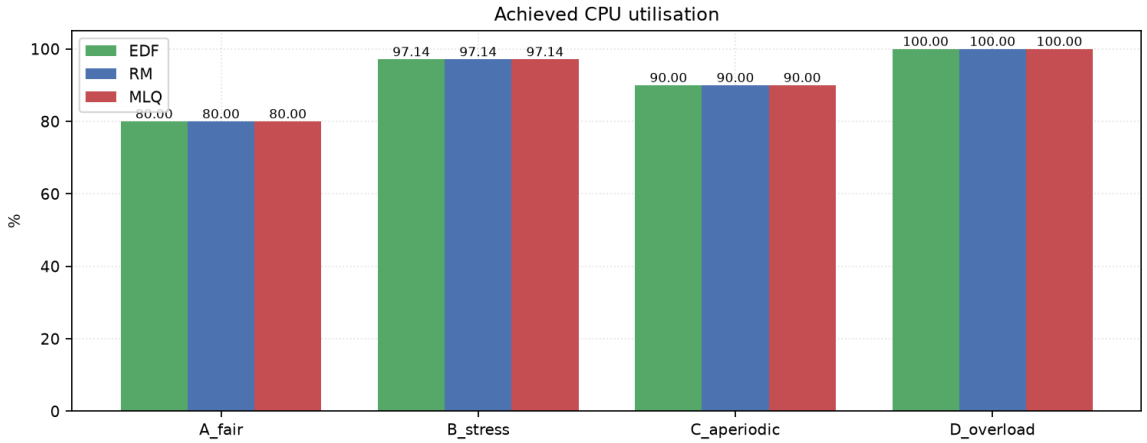


Figure 29. Achieved CPU utilisation in the four scenarios

- In scenarios A and B the utilisation equals the offered load: no time is wasted.
- In scenario C the utilisation is 90% with 3 idle ticks, because the work items are one-shot.
- In scenario D the utilisation saturates at 100% while the offered load is 123.33%.
- That gap is exactly the work that could not be done, and it appears as missed deadlines.
- High utilisation under overload is not a sign of health: it is a sign of full saturation.

Limitation: In this model a context switch costs nothing, so utilisation is not reduced by overhead. It must not be confused with the utilisation of a physical system.

Comprehensive comparison table

Criterion	MLQ	EDF	RM
Priority type	static, by queue	dynamic, by deadline	static, by period
Needs a deadline	no	yes	no
Needs a period	no	no	yes
Analytic guarantee	none	yes (demand criterion)	yes (response-time analysis)
Optimality	none	optimal among dynamic priorities	optimal among fixed priorities
Predictability	medium	good	excellent
Fairness	low	high	low
Starvation risk	high	low	high
Scheduling overhead	low	medium	low
Behaviour under overload	protects the top	spreads the failure	protects the top
Best suited for	classified general-purpose work	dynamic real-time tasks	fixed periodic tasks

Table 30. Master comparison table of the three algorithms

Analysis of the results

- On a real-time workload at moderate utilisation all three succeed; the difference is quality, not correctness.
- At high utilisation only EDF has a guarantee; RM and MLQ failed the exact test and the simulator agreed.
- On a general-purpose workload MLQ is inherently more suitable because it needs no parameters that do not exist.
- Under overload, RM and MLQ protect critical tasks while EDF spreads the failure.
- The overhead of EDF was lower than that of RM in the pressure scenario, contrary to the usual claim.

Why EDF had less overhead here

- In scenario B, EDF performed only 3 preemptions while RM performed 15.
- The reason is that the RM priority repeatedly displaces a running job in favour of the faster task.
- EDF lets the job with the nearer deadline continue and therefore avoids unnecessary switches.
- So the familiar claim that EDF always costs more overhead was not confirmed on this workload.

Measurement limit: Overhead here is measured only by counting context switches, and each switch is assumed to cost nothing. On a real system the cost of recomputing EDF priorities would have to be added and might change the conclusion.

Advantages and disadvantages of each algorithm

EDF

Aspect	Explanation
Advantage	optimal among dynamic priorities and a simple guarantee for implicit deadlines
Advantage	high fairness and lower jitter
Disadvantage	requires a deadline for every task
Disadvantage	spreads failure under overload (domino effect)
Disadvantage	more complex to implement and sensitive to deadline errors

Table 31. EDF: advantages and disadvantages

RM

Aspect	Explanation
Advantage	simple, cheap and optimal among fixed priorities
Advantage	excellent predictability and isolation of critical tasks
Disadvantage	loses schedulable sets at high utilisation
Disadvantage	low-priority tasks may starve
Disadvantage	meaningless for aperiodic work

Table 32. RM: advantages and disadvantages

MLQ

Aspect	Explanation
Advantage	policy separation and explicit priority control
Advantage	needs no real-time parameters
Disadvantage	no analytic guarantee for deadlines
Disadvantage	output quality depends on the manual queue assignment
Disadvantage	starvation risk for the lower queues

Table 33. MLQ: advantages and disadvantages

Applications, limitations and assumptions

Algorithm	Suitable application	Example
EDF	real-time tasks with varied deadlines	flight control, robotics, media streaming
RM	periodic tasks with fixed priorities	industrial control, periodic sensors
DM	tasks whose deadline is shorter than the period	fast sampling systems
MLQ	separating general-purpose workload classes	time-sharing operating systems, databases

Table 34. Application domains of each algorithm

Limitations of this study

- The scenarios are synthetic and small so that hand checking stays possible.
- The model is single-processor; the results do not carry over to multiprocessors.
- The context-switch cost is assumed to be zero.
- No blocking, resource sharing or priority inversion is modelled.
- The worst-case execution time is assumed to be known and constant.
- The simulation horizon is finite; the definitive guarantee comes from the analytic tests.

Suggestions for further work

- Add semaphore blocking and analyse priority inversion.
- Extend to multiprocessors and compare with migration-based algorithms.
- Use traced real workloads instead of synthetic scenarios.
- Study EDF combined with a protection policy for critical tasks.

Conclusion

This exercise showed that comparing MLQ with EDF and RM is only valid when the workload, the timing model and the measurement code are shared and when the different nature of the domains is acknowledged. In the fair scenario at a utilisation of 0.800 all three algorithms met every deadline, and EDF showed a slight quality advantage with an average response time of 3.000 and a jitter of 2.891.

In the high-pressure scenario at a utilisation of 0.9714, response-time analysis predicted that RM is infeasible and the simulator recorded exactly the three deadline misses that the analysis implies, while EDF ran without a miss. The randomised study over 2329 task sets showed that in the utilisation band 0.90 to 1.00 only 17.8% of the sets are feasible under RM against 100% under EDF.

- RM fails at high utilisation while EDF succeeds on the same load.
- MLQ can reproduce RM behaviour on a real-time load, but without an analytic guarantee.
- The price of predictability, measured in context switches, was higher for RM than for EDF here.
- Under overload, RM and MLQ protect critical tasks and EDF distributes the failure.

Key message: No algorithm is universally better. The correct choice is about matching the tool to the problem: EDF or RM when you have deadlines, MLQ when you have distinct workload classes.

References

Primary references

- Liu, C. L., and Layland, J. W. — Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment — Journal of the ACM, 1973. (source of the RM sufficient condition)
- Joseph, M., and Pandya, P. — Finding Response Times in a Real-Time System — The Computer Journal, 1986. (source of response-time analysis)
- Baruah, S. K., Howell, R. R., and Rosier, L. E. — Algorithms and Complexity Concerning the Preemptive Scheduling of Periodic Real-Time Tasks — Information and Computation, 1990. (source of the processor-demand criterion)
- Buttazzo, G. C. — Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications.
- Silberschatz, A., Galvin, P. B., and Gagne, G. — Operating System Concepts — chapters on CPU scheduling and real-time scheduling.

Supplementary references

- Tanenbaum, A. S., and Bos, H. — Modern Operating Systems — scheduling and real-time sections.
- Stallings, W. — Operating Systems: Internals and Design Principles — real-time scheduling chapter.
- Lehoczky, J., Sha, L., and Ding, Y. — The Rate Monotonic Scheduling Algorithm: Exact Characterization and Average Case Behavior.
- Documentation of pytest, matplotlib and reportlab for the implementation tooling.

Citation status: The entries above are the standard references of the operating systems and real-time systems courses. If the instructor requires exact years, volume or page numbers, the copies available to the student must be checked and the citations updated.

Appendix: outputs and requirement mapping

File	Description
output/A_fair_EDF.txt	Gantt chart and metrics of EDF in scenario A
output/A_fair_RM.txt	Gantt chart and metrics of RM in scenario A
output/A_fair_MLQ.txt	Gantt chart and metrics of MLQ in scenario A
output/B_stress_*.txt	results of the three algorithms in the pressure scenario
output/C_aperiodic_*.txt	results in the aperiodic scenario
output/D_overload_*.txt	results in the overload scenario
output/experiments.json	every number in machine-readable form
output/experiments_summary.txt	text summary of all experiments
output/pytest_result.txt	output of the test run
diagrams/gantt_*.png	twelve Gantt charts
diagrams/chart_*.png	eight comparison and analysis charts

Table 35. Output inventory of exercise 2

Evaluation item	Status	Where
EDF and RM introduction	complete	algorithm pages
Real-time versus general-purpose	complete	fundamentals page
Preemptive comparison	complete	fundamental properties page
Liu and Layland condition	complete	exact RM test page
EDF schedulability tests	complete	processor-demand page
Numerical examples for all three	complete	scenarios A to D
Comprehensive comparison table	complete	master table page
Comparison charts	complete	eight charts
Persian and English versions	complete	COMPARISON_FA and COMPARISON_EN
DOCX and PDF versions	complete	docs folder

Table 36. Requirement mapping for exercise 2 (no grade is assumed)